PRACTICAL – 1

Implementation and Time Analysis of Sorting Algorithms

Aim:

To implement and analyze the time complexity of Bubble sort, Selection sort, Insertion sort, Merge sort, and Quicksort.

Objective:

The objective of this practical is to implement and compare the performance of **Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort**. We will analyze the time complexity and execution time of each algorithm to understand their efficiency and identify the most suitable algorithm for different applications.

1.Bubble Sort

Algorithm:

1. Start
2. For i = 0 to n - 2
3. For j = 0 to n - i - 2
4. If A[j] > A[j + 1]
5. Swap A[j] and A[j + 1]
6. End If
7. End For
8. End For
9. Stop

Code:

```
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
   bool swapped;
   for (int i = 0; i < n - 1; i++) {
      swapped = false;
      for (int j = 0; j < n - i - 1; j++) {
         if (arr[j] > arr[j + 1]) {
            // Swap adjacent elements
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
            swapped = true;
         }
```

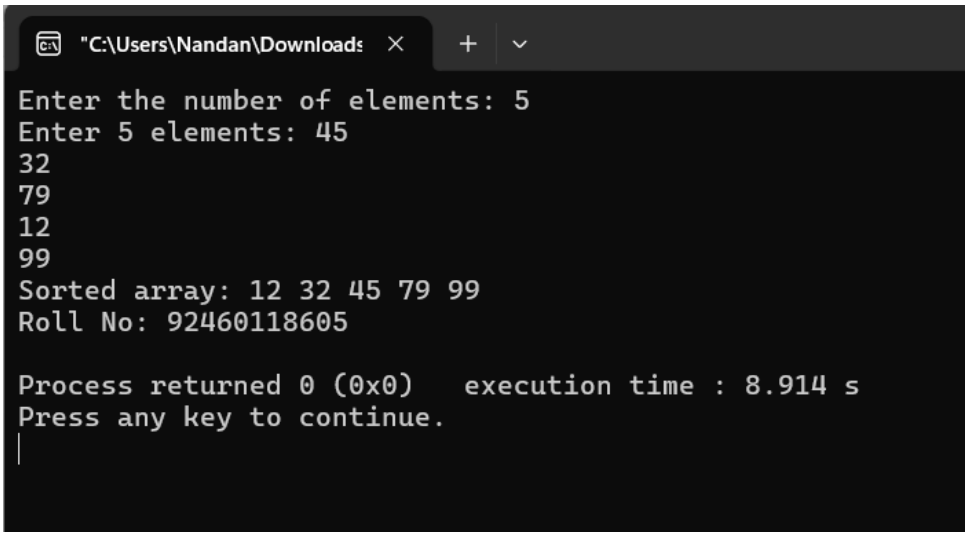
```
        }
        // Stop if no swaps occurred
        if (!swapped)
            break;
    }
}
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}
int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    bubbleSort(arr, n);

    cout << "Sorted array: ";
    printArray(arr, n);

    // Enter your roll number
    cout << "Roll No: 9246018605" << endl;

    delete[] arr;
    return 0;
}
```

Output:



Time Complexity:

Case	Time Complexity
Best Case	O(n)
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space Complexity	O(1)

2.Selection Sort

Algorithm:

1. Start
2. For i = 0 to n - 2
3. Set min = i
4. For j = i + 1 to n - 1
5. If A[j] < A[min]
6. Set min = j
7. End If
8. End For
9. Swap A[i] and A[min]
10. End For
11. Stop

Code:

```
#include <iostream>
using namespace std;

void selectionSort(int arr[], int n) {
   for (int i = 0; i < n - 1; i++) {
      int min_idx = i;
      for (int j = i + 1; j < n; j++) {
         if (arr[j] < arr[min_idx]) {
            min_idx = j;
         }
      }
      if (min_idx != i) {
         int temp = arr[i];
         arr[i] = arr[min_idx];
         arr[min_idx] = temp;
      }
   }
}
```

```
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

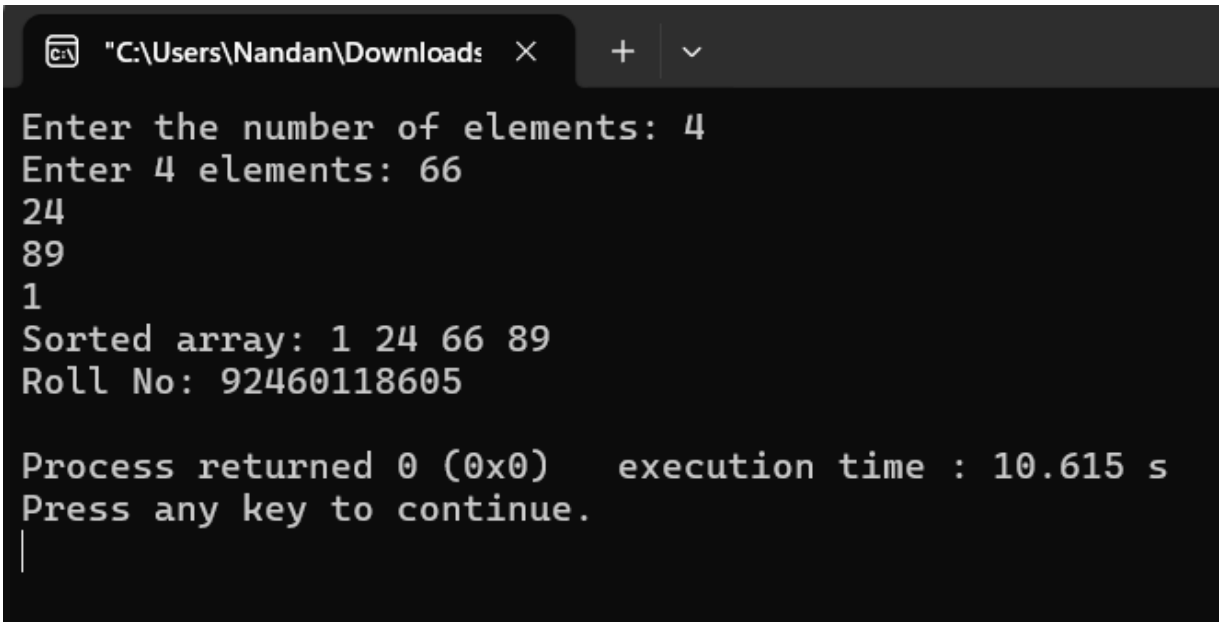
    selectionSort(arr, n);

    cout << "Sorted array: ";
    printArray(arr, n);

    // Enter your roll number
    cout << "Roll No: 92460118605" << endl;

    delete[] arr;
    return 0;
}
```

Output:



Time Complexity:

Case	Time Complexity
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space Complexity	O(1)

3.Insertion Sort

Algorithm:

1. Start
2. For i = 1 to n - 1
3. Set key = A[i]
4. Set j = i - 1
5. While j >= 0 and A[j] > key
6. Set A[j + 1] = A[j]
7. Set j = j - 1
8. End While
9. Set A[j + 1] = key
10. End For
11. Stop

Code:

```
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n) {
   for (int i = 1; i < n; i++) {
      int key = arr[i];
      int j = i - 1;

      while (j >= 0 && arr[j] > key) {
         arr[j + 1] = arr[j];
         j = j - 1;
      }
      arr[j + 1] = key;
   }
}

void printArray(int arr[], int n) {
   for (int i = 0; i < n; i++)
      cout << arr[i] << " ";
   cout << endl;
```

```
}

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

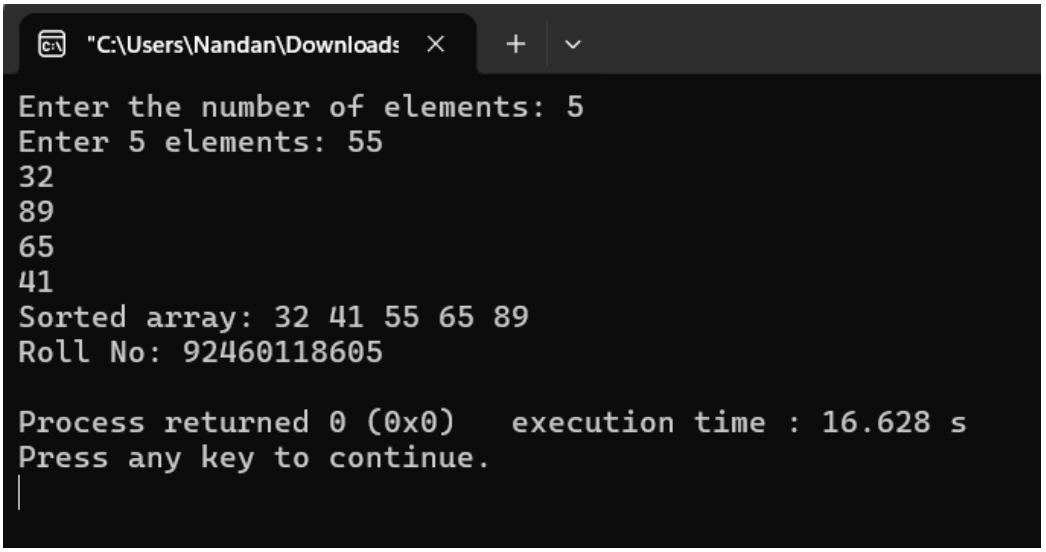
    insertionSort(arr, n);

    cout << "Sorted array: ";
    printArray(arr, n);

    // Enter your roll number
    cout << "Roll No: 92460118605" << endl;

    delete[] arr;
    return 0;
}
```

Output:



Time Complexity:

Case	Time Complexity
Best Case	O(n)
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space Complexity	O(1)

4.Merge Sort

Algorithm:

1. Start
2. If low < high
3. Set mid = (low + high) / 2
4. MergeSort(A, low, mid)
5. MergeSort(A, mid + 1, high)
6. Merge(A, low, mid, high)
7. End If
8. Stop

Code:

```
#include <iostream>
using namespace std;

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int* L = new int[n1];
    int* R = new int[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];

    delete[] L;
    delete[] R;
}

void mergeSort(int arr[], int left, int right) {
```

```
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

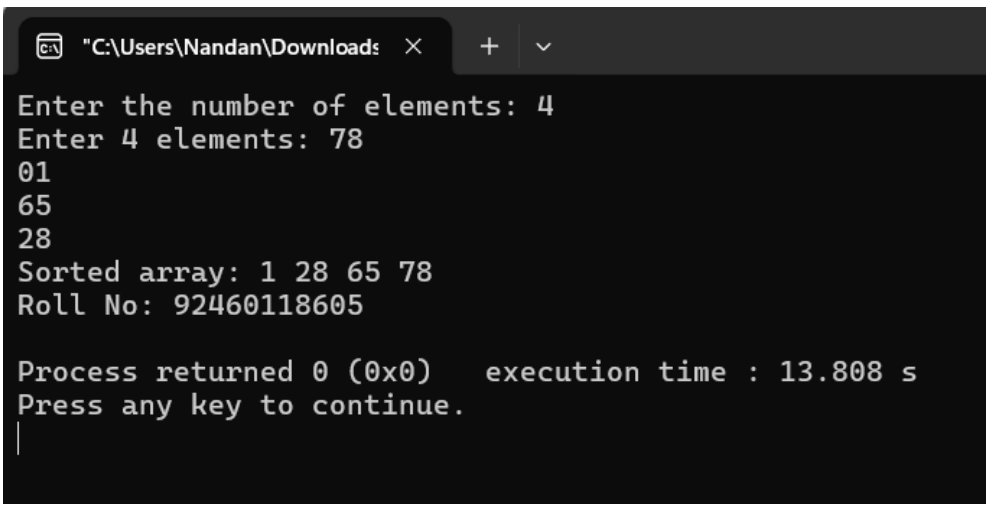
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    mergeSort(arr, 0, n - 1);
    cout << "Sorted array: ";
    printArray(arr, n);

    // Enter your roll number
    cout << "Roll No: 92460118605" << endl;

    delete[] arr;
    return 0;
}
```

Output:



Time Complexity:

Case	Time Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n log n)
Space Complexity	O(n)

5.Quick Sort

Algorithm:

1. Start
2. If low < high
3. Set pivot = Partition(A, low, high)
4. QuickSort(A, low, pivot - 1)
5. QuickSort(A, pivot + 1, high)
6. End If
7. Stop

Code:

```
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high) {
   int pivot = arr[high];
   int i = (low - 1);

   for (int j = low; j <= high - 1; j++) {
      if (arr[j] < pivot) {
         i++;
         int temp = arr[i];
         arr[i] = arr[j];
         arr[j] = temp;
      }
   }
   int temp = arr[i + 1];
   arr[i + 1] = arr[high];
   arr[high] = temp;

   return (i + 1);
}

void quickSort(int arr[], int low, int high) {
```

```
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

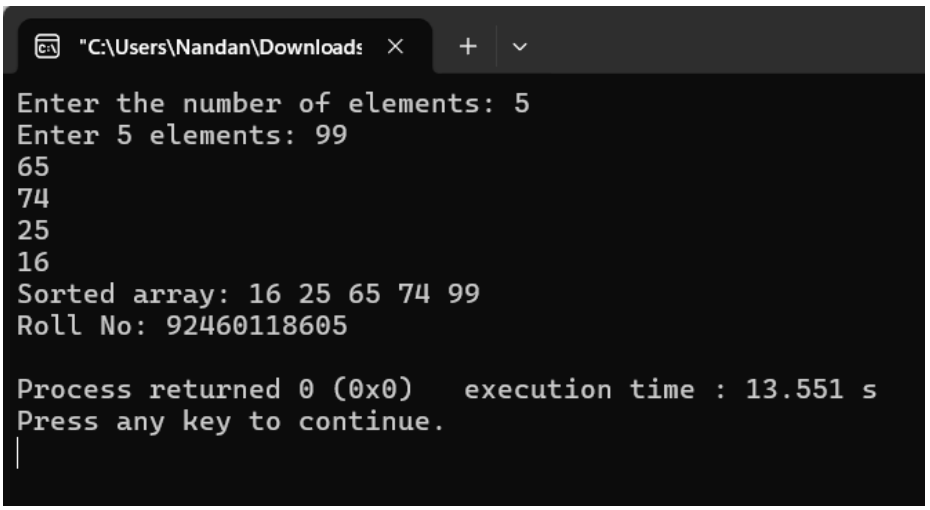
int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    quickSort(arr, 0, n - 1);
    cout << "Sorted array: ";
    printArray(arr, n);

    // Enter your roll number
    cout << "Roll No: 92460118605" << endl;

    delete[] arr;
    return 0;
}
```

Output:



Time Complexity:

Case	Time Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	$O(n^2)$
Space Complexity	O(log n)